SOFTWARE ENGINEERING CANDIDATE GUIDE

Comprehensive Interview Preparation for Recent Graduates

Part 1: High-Impact Resume Points

As a recent graduate, your resume needs to bridge the gap between theoretical knowledge and practical engineering. Use these bullet points to highlight your transition from algorithmic problem-solving to building robust system architectures.

Core Projects & Experience

- **AI Systems Engineering:** Architected and deployed "SystemOrbit," an autonomous AI support agent utilizing Python and FastAPI to process natural language queries and execute backend workflows.
- **Retrieval-Augmented Generation (RAG):** Integrated ChromaDB as a local vector database, enabling semantic search capabilities over company documentation to eliminate LLM hallucinations.
- **Agentic Tool-Calling Workflows:** Engineered a two-pass execution loop allowing the LLM to dynamically trigger Python functions (e.g., automated Jira ticket creation) based on inferred user intent.
- **Algorithmic Optimization:** Consistently solve complex data structure and algorithmic challenges on platforms like LeetCode and CodeChef, applying advanced logic to optimize runtime and memory complexity.
- **Security & Environment Management:** Navigated CTF challenges utilizing tools like aprisolve; proficient in Windows PowerShell for local environment configuration, virtual environments, and package management.
- **Full-Stack Versatility:** Strong foundational knowledge in core programming languages including Python and Java, with experience designing RESTful API endpoints.

Part 2: Interview Questions & Answers

This section covers behavioral and technical questions, scaling from basic conversational topics to advanced system design.

Level 1: Basic & Behavioral

> **Q: Tell me about yourself.**

A: "I am a recent computer science graduate with a deep passion for backend logic and artificial intelligence. My background is heavily rooted in competitive programming—I spend a lot of time on LeetCode refining my algorithmic problem-solving skills. Recently, I've transitioned from solving isolated algorithms to building full-scale architecture. For example, I just finished building an AI agent project using Python, FastAPI, and vector databases. I'm eager to bring my optimization mindset into a production environment."

> **Q: What is your strongest programming language, and why?**

A: "Python is my strongest language. I love its versatility—it allows me to quickly prototype complex algorithms for LeetCode challenges, but it's also the absolute industry standard for AI and backend systems. I'm very comfortable using Python for everything from basic scripting and data manipulation to standing up web servers with FastAPI and building AI agent loops. I also have a strong foundation in Java, which I appreciate for its strict object-oriented structure."

Level 2: Intermediate Technical

> **Q: Explain the difference between a traditional Hash Map and a Vector Database.**

A: "A Hash Map is excellent for exact O(1) matching. If I need to look up user ID '123', a Hash Map finds that exact string instantly. However, it fails with semantics. A Vector Database, like ChromaDB, translates text into high-dimensional numerical arrays (embeddings). This allows us to search by mathematical proximity or 'meaning'. If a user asks for 'paid time off', a Vector Database knows it mathematically matches a document about 'vacation days', whereas a Hash Map would return zero results."

Q: How do you handle environment variables and sensitive data in your projects?

A: "I never hardcode sensitive data like API keys or database passwords into my source code. I use a `.env` file to store these secrets locally, and I ensure that file is added to my `.gitignore` so it is never pushed to version control. In Python, I use libraries like `python-dotenv` to securely load these variables into the operating system's memory at runtime."

Level 3: Advanced Architecture & Agents

Q: Can you explain how 'Agentic Tool Calling' works under the hood in an LLM application?

A: "It fundamentally relies on a two-pass execution loop. First, you define a standard Python function and pass its schema (name, description, arguments) to the LLM. In the first pass, the user asks a question. The LLM recognizes it needs data and halts text generation, returning a structured JSON payload requesting the tool. My application intercepts this, executes the actual Python function (like creating a Jira ticket), and captures the result. In the second pass, I append that result to the message history and send it back to the LLM, which then formulates a final, natural language summary for the user."

Q: What is an ASGI server, and how does a web framework like FastAPI handle requests?

A: "Unlike a standard script that runs and terminates, a web server runs an infinite loop listening for incoming HTTP requests. FastAPI is a modern web framework that defines the API routes (the endpoints), while an ASGI server like Uvicorn actually manages the network connections and keeps the application awake. When a JSON payload is POSTed to an endpoint, FastAPI validates the data using Pydantic models, executes the internal logic—like querying a database or calling an AI model—and returns the response."

Part 3: The 10-Minute Panel Presentation Script

Use this script when asked to present a project or talk extensively about your background. It is designed to be conversational, engaging, and highly technical.

INTRODUCTION (MINUTES 0-2)

"Hi everyone, thank you so much for the opportunity to speak with you today. I'm a recent graduate with a very strong foundation in algorithmic logic and backend development.

For a long time, my primary focus was competitive programming. I loved diving into LeetCode, optimizing time and space complexity, and understanding data structures inside and out. But recently, I made a conscious pivot. I realized that being a great software engineer isn't just about solving isolated puzzles—it's about gluing those pieces together to build real-world architecture. I wanted to build systems that take action."

THE PROJECT HOOK (MINUTES 2-5)

"To bridge this gap, I decided to build a project called 'SystemOrbit'. My goal was to build an internal support assistant. But I didn't want to build just a standard chatbot wrapper around an API; I wanted to build a true Agent—an application that reads private company documents and takes real-world actions.

I built the backend using Python and FastAPI. The first major challenge I tackled was giving the AI context. Standard LLMs have a knowledge cutoff, so I implemented a Retrieval-Augmented Generation (RAG) pipeline. I set up ChromaDB, a local vector database, to store dummy company policies. This way, when a user asks a question, my system performs a semantic mathematical search, retrieves the right document, and injects it into the prompt."

THE TECHNICAL DEEP DIVE (MINUTES 5-8)

"The part of the project I am most proud of is the Agentic workflow. I wanted the AI to have 'hands'.

I engineered a two-pass tool-calling loop. I wrote Python functions—for instance, a mock function to create an IT Jira ticket. I passed the schema of this function to a local AI engine running via Ollama. Now, if a user types, 'My laptop caught on fire,' the AI doesn't just give advice. It autonomously decides to pause, requests the tool, passes in the exact argument 'laptop fire replacement', executes the Python function, and returns a ticket number to the user.

Connecting the web framework, the vector database, and the agentic loop into a single seamless API route was a massive 'aha' moment for me in understanding systems engineering."

THE CLOSE (MINUTES 8-10)

> "Ultimately, this project taught me how to handle asynchronous operations, manage environment variables securely, and structure a backend application.
>
> What I bring to this team is the rigorous, optimization-first mindset from my competitive programming background, combined with a proven ability to independently learn and deploy modern architectural patterns. I'm incredibly eager to learn from the senior engineers here and start contributing to production code.
>
> I'd be happy to dive deeper into any of the specific libraries I used or talk more about my transition into backend engineering. Thank you."